
A Bunched Approach to Directed HoTT

Louise Leclerc

Samuel Mimram

September 17, 2026

Outline

- ① Homotopy type theory
- ② Higher categories
- ③ Simplicial type theory
- ④ The problem of hom types
- ⑤ The bunched solution
- ⑥ From theory to application

Presentation

(Homotopy) Type Theory:

Presentation

(Homotopy) Type Theory:

- Mathematical language (like Set Theory)

Presentation

(Homotopy) Type Theory:

- Mathematical language (like Set Theory)
- Constructive flavor ($\neq$ Classical Set Theory)

Presentation

(Homotopy) Type Theory:

- Mathematical language (like Set Theory)
- Constructive flavor ($\neq$ Classical Set Theory)
- Types behaves as spaces.

Presentation

(Homotopy) Type Theory:

- Mathematical language (like Set Theory)
- Constructive flavor ($\neq$ Classical Set Theory)
- Types behaves as spaces.
- Synthetic tool for higher algebra, homotopy theory...

Presentation

(Homotopy) Type Theory:

- Mathematical language (like Set Theory)
- Constructive flavor ($\neq$ Classical Set Theory)
- Types behaves as spaces.
- Synthetic tool for higher algebra, homotopy theory...
- Suitable for formalizing mathematics / homotopy theory

Presentation

(Homotopy) Type Theory:

- Mathematical language (like Set Theory)
- Constructive flavor ($\neq$ Classical Set Theory)
- Types behaves as spaces.
- Synthetic tool for higher algebra, homotopy theory...
- Suitable for formalizing mathematics / homotopy theory
- **Goal:** replace spaces by *directed spaces*

Intuitionistic Type Theory

MLTT:

Intuitionistic Type Theory

MLTT:

- MARTIN-LÖF, 1972.

Intuitionistic Type Theory

MLTT:

- MARTIN-LÖF, 1972.
- conversely to Set Theory, no need for first order logic.

Intuitionistic Type Theory

MLTT:

- MARTIN-LÖF, 1972.
- conversely to Set Theory, no need for first order logic.
- “basic types” and “constructors” as building blocks.

Basic Types

- **empty type:** $\perp$ or $\mathbb{0}$.
no native element or constructor.

Basic Types

- **empty** type: $\perp$ or $\mathbb{0}$.
no native element or constructor.
- **unit** type: $\top$ or $\mathbb{1}$.
one native element, $*$.
 $* : \mathbb{1}$.

Basic Types

- **empty** type: $\perp$ or $\mathbb{0}$.
no native element or constructor.
- **unit** type: $\top$ or $\mathbb{1}$.
one native element, $*$.
 $* : \mathbb{1}$.
- **booleans** type, Bool or $\mathbb{2}$.
two native elements of this type, false and true or 0 and 1.
 $0 : \mathbb{2}$ or $1 : \mathbb{2}$.

Basic Types

- **empty type**: $\perp$ or $\mathbb{0}$.
no native element or constructor.
- **unit type**: $\top$ or $\mathbb{1}$.
one native element, $*$.
 $* : \mathbb{1}$.
- **booleans type**, **Bool** or $\mathbb{2}$.
two native elements of this type, false and true or 0 and 1.
 $0 : \mathbb{2}$ or $1 : \mathbb{2}$.
- **natural numbers type**, $\mathbb{N}$.
 $0 : \mathbb{N}$, $\text{succ} : \mathbb{N} \rightarrow \mathbb{N}$.
 $n : \mathbb{N} \rightsquigarrow \text{succ}(n) : \mathbb{N}$.
eg., $\text{succ}(\text{succ}(\text{succ}(0))) : \mathbb{N}$.

Type operations

- Arrow type: $A \rightarrow B$

Type operations

- Arrow type: $A \rightarrow B$
- Cartesian product type: $A \times B$

Type operations

- Arrow type: $A \rightarrow B$
- Cartesian product type: $A \times B$
- Sum type: $A + B$

Hierarchy of Universes

- Tarski (or Grothendieck) Universes. $\mathcal{U}$ is the “type of types”
- $\mathcal{U}$ closed under $\times$, $+$ and $\rightarrow$.
- Having $\mathcal{U} : \mathcal{U}$ leads to paradoxes

$$\mathcal{U}_0 : \mathcal{U}_1 : \mathcal{U}_2 : \dots$$

recursors

How to map *out* of a given type ?

recursors

How to map *out* of a given type ?

- $\mathbb{0}$:
for any A , $\mathbf{rec}_{\mathbb{0}, A} : \mathbb{0} \rightarrow A$

recursors

How to map *out* of a given type ?

- $\mathbb{0}$:
for any A , $\text{rec}_{\mathbb{0}, A} : \mathbb{0} \rightarrow A$
- $\mathbb{1}$:
for any A and $a : A$, $\text{rec}_{\mathbb{1}, A, a} : \mathbb{1} \rightarrow A$
 $\text{rec}_{\mathbb{1}, A, a} (*) \equiv a$.

recursors

How to map *out* of a given type ?

- $\mathbb{0}$:
for any A , $\text{rec}_{\mathbb{0}, A} : \mathbb{0} \rightarrow A$
- $\mathbb{1}$:
for any A and $a : A$, $\text{rec}_{\mathbb{1}, A, a} : \mathbb{1} \rightarrow A$
 $\text{rec}_{\mathbb{1}, A, a}(\ast) \equiv a$.
- $\mathbb{2}$:
for any A , and $a_0 : A$, $a_1 : A$, $\text{rec}_{\mathbb{2}, A, a_0, a_1} : \mathbb{2} \rightarrow A$
 $\text{rec}_{\mathbb{2}, A, a_0, a_1}(0) \equiv a_0$ and $\text{rec}_{\mathbb{2}, A, a_0, a_1}(1) \equiv a_1$.

recursors

- $\mathbb{N}$:
for any $A, a_0 : A$ and $f : \mathbb{N} \rightarrow A \rightarrow A$, $\varphi \equiv \text{rec}_{\mathbb{N}, A, a_0, f} : \mathbb{N} \rightarrow A$.
 $\varphi(0) \equiv a_0$ and $\varphi(\text{succ}(n)) \equiv f(n)(\varphi(n))$.

recursors

- $\mathbb{N}$:
for any $A, a_0 : A$ and $f : \mathbb{N} \rightarrow A \rightarrow A$, $\varphi \equiv \text{rec}_{\mathbb{N}, A, a_0, f} : \mathbb{N} \rightarrow A$.
 $\varphi(0) \equiv a_0$ and $\varphi(\text{succ}(n)) \equiv f(n)(\varphi(n))$.
- $A \times B$:
for any types A, B , and $f : A \rightarrow B \rightarrow C$, $\text{rec}_{A \times B, C, f} : A \times B \rightarrow C$
 $\text{rec}_{A \times B, C, f}(a, b) \equiv f(a)(b)$.

recursors

- $\mathbb{N}$:
for any $A, a_0 : A$ and $f : \mathbb{N} \rightarrow A \rightarrow A$, $\varphi \equiv \text{rec}_{\mathbb{N}, A, a_0, f} : \mathbb{N} \rightarrow A$.
 $\varphi(0) \equiv a_0$ and $\varphi(\text{succ}(n)) \equiv f(n)(\varphi(n))$.
- $A \times B$:
for any types A, B , and $f : A \rightarrow B \rightarrow C$, $\text{rec}_{A \times B, C, f} : A \times B \rightarrow C$
 $\text{rec}_{A \times B, C, f}(a, b) \equiv f(a)(b)$.
- $A + B$:
for any types $A, B, f : A \rightarrow C$ and $g : B \rightarrow C$, $\text{rec}_{A+B, C, f, g} : A + B \rightarrow C$
 $\text{rec}_{A+B, C, f, g}(\text{inl}(a)) \equiv f(a)$ and $\text{rec}_{A+B, C, f, g}(\text{inr}(b)) \equiv g(b)$.

More type formers

- Π -types:

$$A : \mathcal{U}, B : A \rightarrow \mathcal{U} \quad \rightsquigarrow \quad \prod_{a:A} B(a) : \mathcal{U}.$$

$$f : \prod_{a:A} B(a), x : A \quad \rightsquigarrow \quad f(x) : B(x).$$

More type formers

- Π -types:

$$\begin{aligned} A : \mathcal{U}, B : A \rightarrow \mathcal{U} &\rightsquigarrow \prod_{a:A} B(a) : \mathcal{U}. \\ f : \prod_{a:A} B(a), x : A &\rightsquigarrow f(x) : B(x). \end{aligned}$$

- Σ -types:

$$\begin{aligned} A : \mathcal{U}, B : A \rightarrow \mathcal{U} &\rightsquigarrow \sum_{a:A} B(a) : \mathcal{U}. \\ x : A, u : B(a) &\rightsquigarrow (x, u) : \sum_{a:A} B(a). \end{aligned}$$

More type formers

- Π -types:

$$\begin{aligned} A : \mathcal{U}, B : A \rightarrow \mathcal{U} &\rightsquigarrow \prod_{a:A} B(a) : \mathcal{U}. \\ f : \prod_{a:A} B(a), x : A &\rightsquigarrow f(x) : B(x). \end{aligned}$$

- Σ -types:

$$\begin{aligned} A : \mathcal{U}, B : A \rightarrow \mathcal{U} &\rightsquigarrow \sum_{a:A} B(a) : \mathcal{U}. \\ x : A, u : B(a) &\rightsquigarrow (x, u) : \sum_{a:A} B(a). \end{aligned}$$

- Identity-types:

$$\begin{aligned} A : \mathcal{U}, x, y : A &\rightsquigarrow x =_A y : \mathcal{U}. \\ x : A &\rightsquigarrow \text{refl}_x : x =_A x. \end{aligned}$$

Curry–Howard

Correspondance between types and logical propositions.

Curry–Howard

Correspondance between types and logical propositions.

- $\rightarrow$ is the implication arrow.

Curry–Howard

Correspondance between types and logical propositions.

- $\rightarrow$ is the implication arrow.
- $\times$ is the conjunction.

Curry–Howard

Correspondance between types and logical propositions.

- $\rightarrow$ is the implication arrow.
- $\times$ is the conjunction.
- $+$ is a constructive disjunction.

Curry–Howard

Correspondance between types and logical propositions.

- $\rightarrow$ is the implication arrow.
- $\times$ is the conjunction.
- $+$ is a constructive disjunction.

Logic $\hookrightarrow$ Algebra

The identity type

- In ordinary maths, $=$ is a proposition. Now it becomes a *type*:

$$A : \mathcal{U}, x, y : A \quad \rightsquigarrow \quad x =_A y : \mathcal{U}$$

The identity type

- In ordinary maths, $=$ is a proposition. Now it becomes a *type*:

$$A : \mathcal{U}, x, y : A \quad \rightsquigarrow \quad x =_A y : \mathcal{U}$$

- Canonical proof: $\text{refl}_x : x =_A x$

The identity type

- In ordinary maths, $=$ is a proposition. Now it becomes a *type*:

$$A : \mathcal{U}, x, y : A \quad \rightsquigarrow \quad x =_A y : \mathcal{U}$$

- Canonical proof: $\text{refl}_x : x =_A x$
- **induction principle**: to prove something for every $p : x = y$, it suffices to treat the case $y \equiv x$, $p \equiv \text{refl}_x$.

The identity type

- In ordinary maths, $=$ is a proposition. Now it becomes a *type*:

$$A : \mathcal{U}, x, y : A \quad \rightsquigarrow \quad x =_A y : \mathcal{U}$$

- Canonical proof: $\text{refl}_x : x =_A x$
- **induction principle**: to prove something for every $p : x = y$, it suffices to treat the case $y \equiv x$, $p \equiv \text{refl}_x$.
- Think of $x = y$ as the type of paths from x to y .

Groupoid structure on types

- **concatenation:** $p : x = y, q : y = z \rightsquigarrow p \cdot q : x = z$

Groupoid structure on types

- **concatenation:** $p : x = y, q : y = z \rightsquigarrow p \cdot q : x = z$
- **inverse:** $p : x = y \rightsquigarrow p^{-1} : y = x$

Groupoid structure on types

- **concatenation:** $p : x = y, q : y = z \rightsquigarrow p \cdot q : x = z$
- **inverse:** $p : x = y \rightsquigarrow p^{-1} : y = x$
- **unit:** $\text{refl}_x \cdot p = p$ and $p \cdot \text{refl}_y = p$

Groupoid structure on types

- **concatenation:** $p : x = y, q : y = z \rightsquigarrow p \cdot q : x = z$
- **inverse:** $p : x = y \rightsquigarrow p^{-1} : y = x$
- **unit:** $\text{refl}_x \cdot p = p$ and $p \cdot \text{refl}_y = p$
- **associativity:** $p \cdot (q \cdot r) = (p \cdot q) \cdot r$

Groupoid structure on types

- **concatenation:** $p : x = y, q : y = z \rightsquigarrow p \cdot q : x = z$
- **inverse:** $p : x = y \rightsquigarrow p^{-1} : y = x$
- **unit:** $\text{refl}_x \cdot p = p$ and $p \cdot \text{refl}_y = p$
- **associativity:** $p \cdot (q \cdot r) = (p \cdot q) \cdot r$
- **inversion:** $p \cdot p^{-1} = \text{refl}_x$ and $p^{-1} \cdot p = \text{refl}_y$

Groupoid structure on types

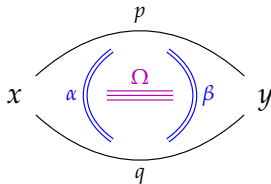
- **concatenation:** $p : x = y, q : y = z \rightsquigarrow p \cdot q : x = z$
- **inverse:** $p : x = y \rightsquigarrow p^{-1} : y = x$
- **unit:** $\text{refl}_x \cdot p = p$ and $p \cdot \text{refl}_y = p$
- **associativity:** $p \cdot (q \cdot r) = (p \cdot q) \cdot r$
- **inversion:** $p \cdot p^{-1} = \text{refl}_x$ and $p^{-1} \cdot p = \text{refl}_y$

Theorem

Every type is equipped with a canonical structure of ∞ -groupoid.

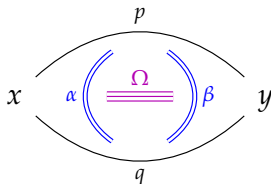
∞ -groupoids

Spaces up to homotopy



∞ -groupoids

Spaces up to homotopy



$$x, y : A$$

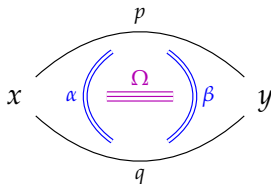
$$p, q : x =_A y$$

$$\alpha, \beta : p =_{x =_A y} q$$

$$\Omega : \alpha =_{p =_{x =_A y} q} \beta$$

∞ -groupoids

Spaces up to homotopy



$$x, y : A \qquad p, q : x =_A y \qquad \alpha, \beta : p =_{x =_A y} q \qquad \Omega : \alpha =_{p =_{x =_A y} q} \beta$$

Formalisation of homotopy theory

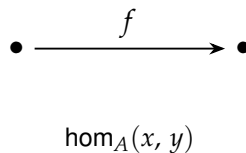
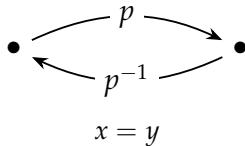
(Feudenthal, long exact sequences, sphere homotopy groups...).

Non-directedness of HoTT

- Paths $x = y$ are always *invertible*.

Non-directedness of HoTT

- Paths $x = y$ are always *invertible*.

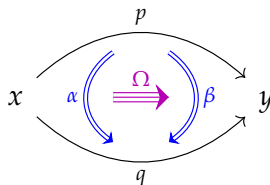


∞ -categories

"Directed" spaces up to homotopy

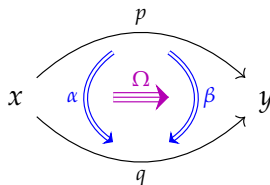
∞ -categories

"Directed" spaces up to homotopy



∞ -categories

"Directed" spaces up to homotopy



$$x, y : A$$

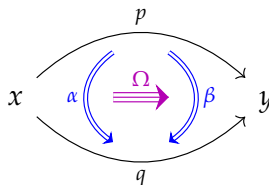
$$p, q : \text{hom}_A(x, y)$$

$$\alpha, \beta : \text{hom}_{\text{hom}_A(x, y)}(p, q)$$

$$\Omega : \text{hom}_{\text{hom}_{\text{hom}_A(x, y)}(p, q)}(\alpha, \beta)$$

∞ -categories

"Directed" spaces up to homotopy



$$x, y : A$$

$$p, q : \text{hom}_A(x, y)$$

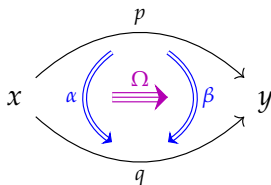
$$\alpha, \beta : \text{hom}_{\text{hom}_A(x, y)}(p, q)$$

$$\Omega : \text{hom}_{\text{hom}_{\text{hom}_A(x, y)}(p, q)}(\alpha, \beta)$$

Rewriting, concurrency,

∞ -categories

"Directed" spaces up to homotopy



$$x, y : A$$

$$p, q : \text{hom}_A(x, y)$$

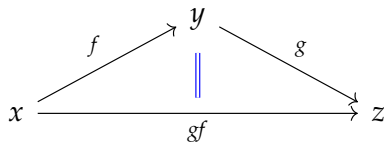
$$\alpha, \beta : \text{hom}_{\text{hom}_A(x, y)}(p, q)$$

$$\Omega : \text{hom}_{\text{hom}_{\text{hom}_A(x, y)}(p, q)}(\alpha, \beta)$$

Rewriting, concurrency,
synthetic manipulation of higher categories.

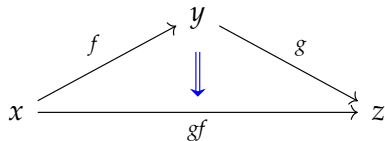
Homotopy coherence

ordinary categories



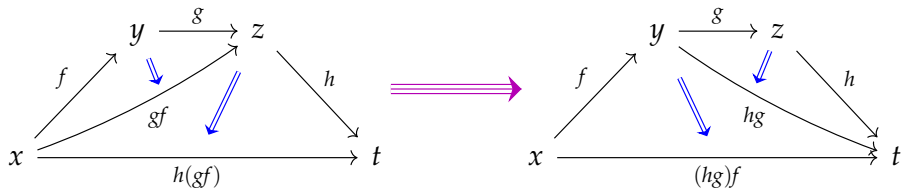
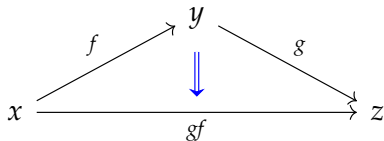
Homotopy coherence

higher categories



Homotopy coherence

higher categories



The nerve construction

Idea: $\text{categories} \leadsto \text{simplicial set}$

The nerve construction

Idea:

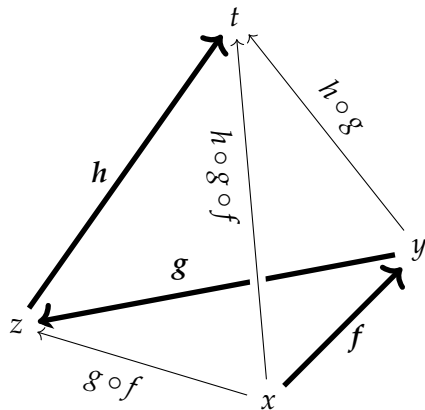
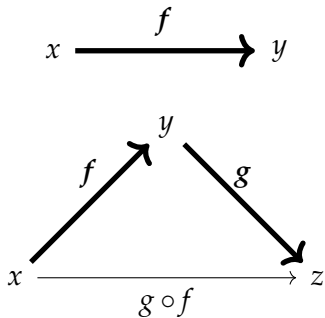
categories $\leadsto$ simplicial set

$$x \xrightarrow{f} y$$

The nerve construction

Idea:

categories $\leadsto$ simplicial set



- *Space* of objects X_0

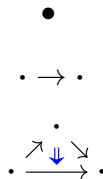
Simplicial spaces

- *Space* of objects X_0
- *Space* of morphisms X_1



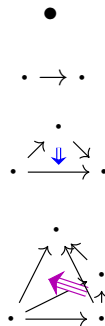
Simplicial spaces

- *Space* of objects X_0
- *Space* of morphisms X_1
- *Space* of composition witness X_2



Simplicial spaces

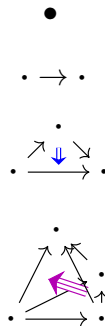
- *Space* of objects X_0
- *Space* of morphisms X_1
- *Space* of composition witness X_2
- *Space* of associativity witness X_3



Simplicial spaces

- *Space* of objects X_0
- *Space* of morphisms X_1
- *Space* of composition witness X_2
- *Space* of associativity witness X_3
- $\vdots$

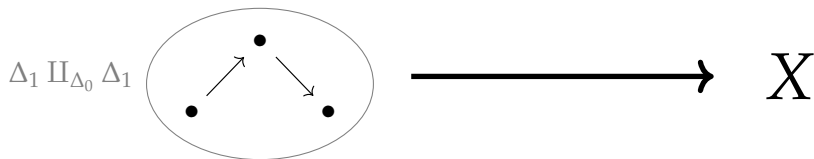
Space valued presheaf $X : \Delta^{\text{op}} \longrightarrow \mathcal{S}$.



Segal spaces

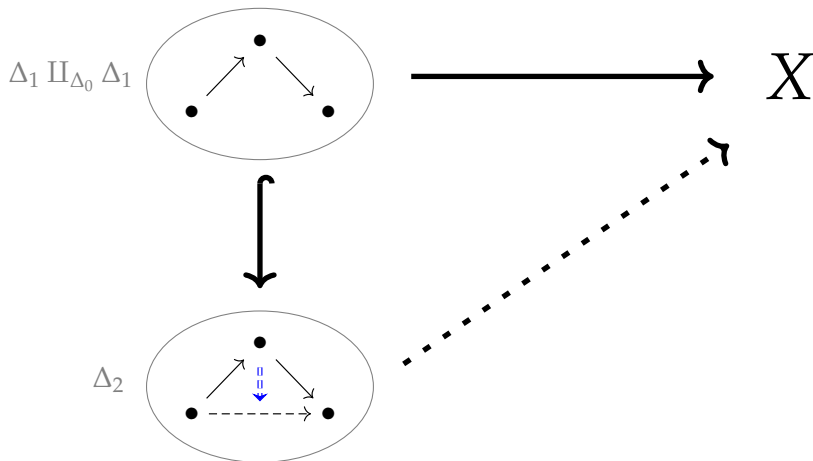
Idea:

encode composability as lifting properties.



Segal spaces

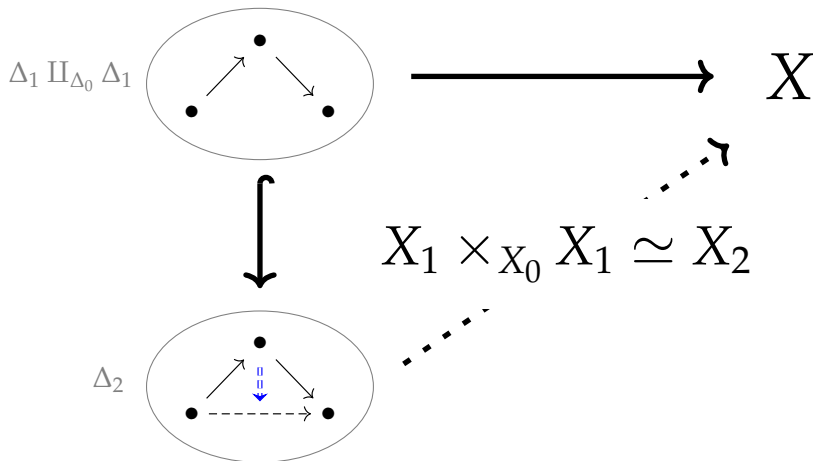
Idea: encode composability as lifting properties.



Segal spaces

Idea:

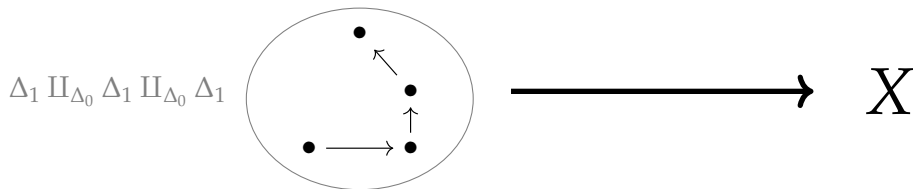
encode composability as lifting properties.



Segal spaces (2)

Idea:

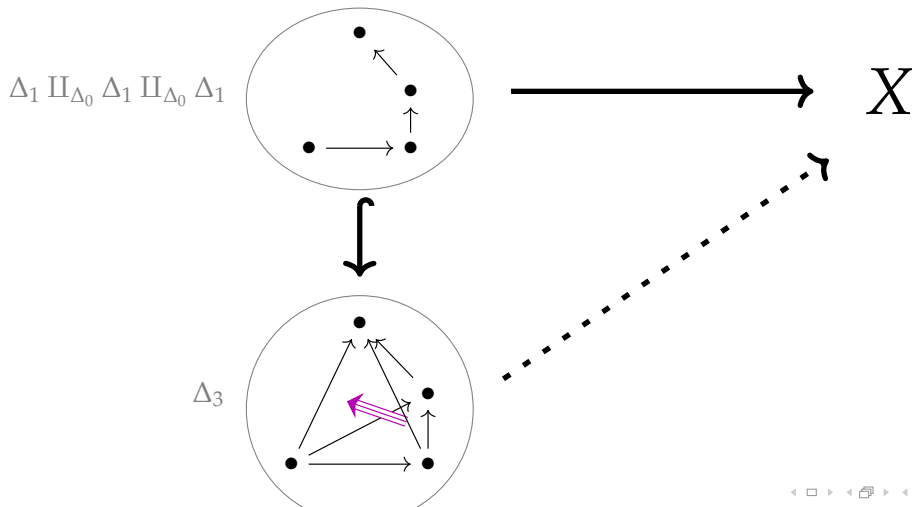
encode composability as lifting properties.



Segal spaces (2)

Idea:

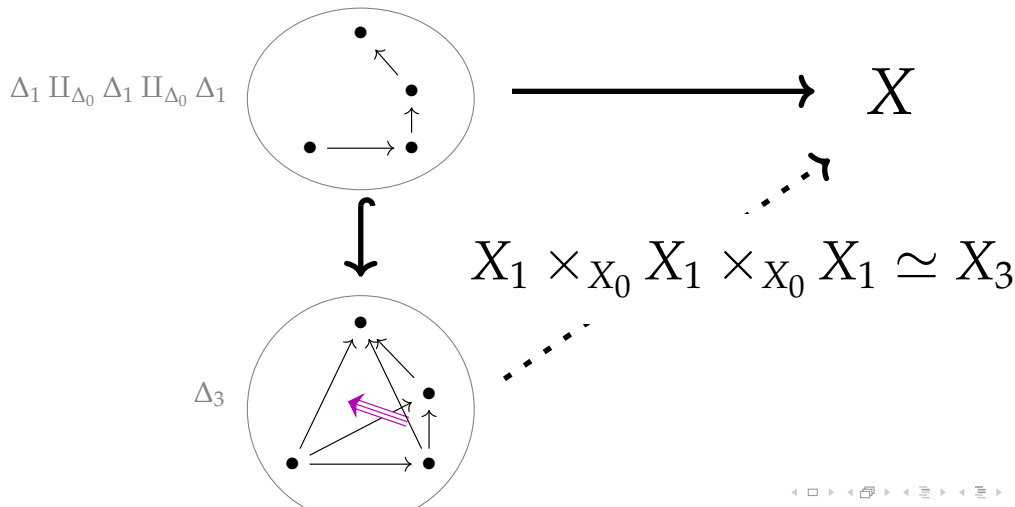
encode composability as lifting properties.



Segal spaces (2)

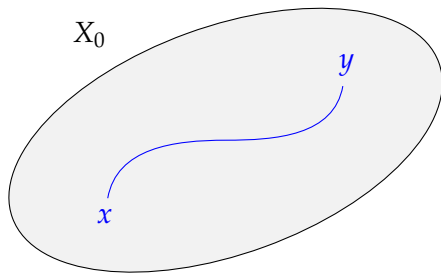
Idea:

encode composability as lifting properties.



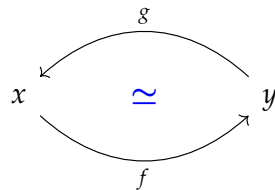
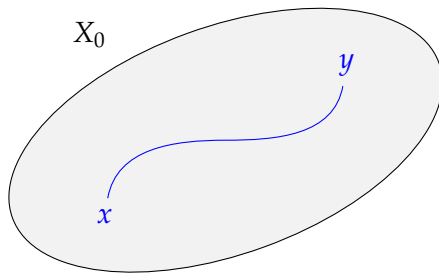
Complete segal spaces

Two coexisting kinds of identifications



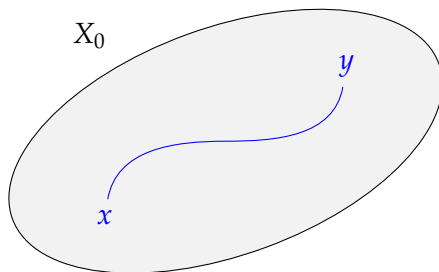
Complete segal spaces

Two coexisting kinds of identifications

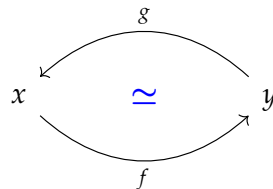


Complete segal spaces

Two coexisting kinds of identifications



$\simeq$



Simplicial type theory

- Riehl, Shulman (2017); Gratzer, Weinberger, Buchholtz...

Simplicial type theory

- Riehl, Shulman (2017); Gratzer, Weinberger, Buchholtz. . .
- Postulate simplices as part of the syntax:

$$\llbracket \mathcal{U} \rrbracket = \mathbf{Psh}(\Delta)$$

Simplicial type theory

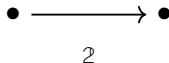
- Riehl, Shulman (2017); Gratzer, Weinberger, Buchholtz...
- Postulate simplices as part of the syntax:

$$\llbracket \mathcal{U} \rrbracket = \mathbf{Psh}(\Delta)$$

- Goal: $(\infty, 1)$ -categories = Complete Segal spaces.

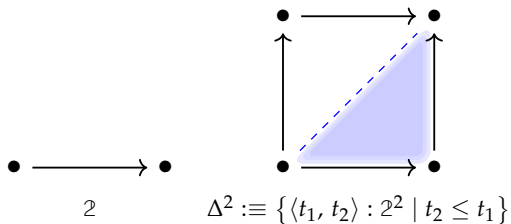
The interval

We add to the syntax a collection of shapes



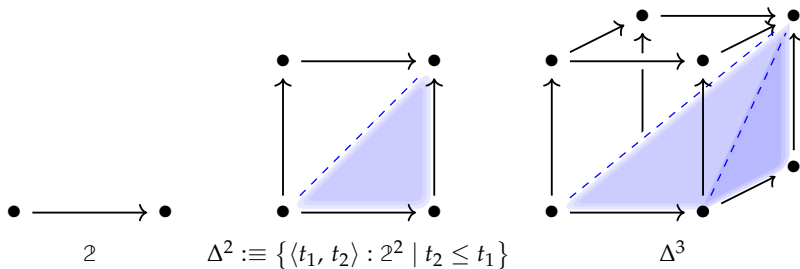
The interval

We add to the syntax a collection of shapes



The interval

We add to the syntax a collection of shapes



Simplicial types

S shape, and X type:

 $S \rightarrow X$ is a type

Simplicial types

S shape, and X type:

$S \rightarrow X$ is a type

$$\mathrm{hom}_X(x, y) = \sum_{f:2 \rightarrow X} (f(0) = x) \times (f(1) = y)$$

Simplicial types

S shape, and X type:

$S \rightarrow X$ is a type

$$\mathrm{hom}_X(x, y) = \sum_{f:2 \rightarrow X} (f(0) = x) \times (f(1) = y)$$

$\forall n, \quad X_n \equiv \Delta_n \rightarrow X$ is a type

Simplicial types

S shape, and X type:

$S \rightarrow X$ is a type

$$\mathrm{hom}_X(x, y) = \sum_{f:2 \rightarrow X} (f(0) = x) \times (f(1) = y)$$

$\forall n, \quad X_n \equiv \Delta_n \rightarrow X$ is a type

So each type give rise to a *simplicial type* !

Segal types

When does (X_n) satisfies the Segal condition ?

Segal types

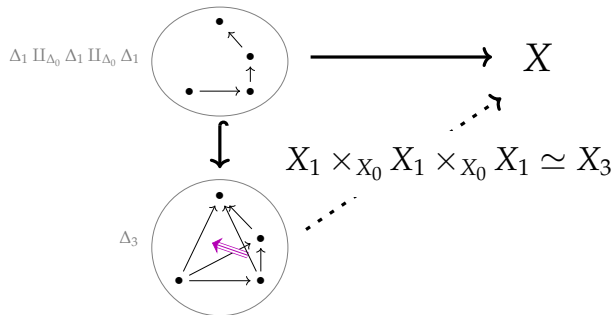
When does (X_n) satisfies the Segal condition ?

$$(\Delta^n \rightarrow X) \xrightarrow{\sim} (\Delta_1 \amalg_{\Delta_0} \amalg \cdots \amalg_{\Delta_0} \Delta_1 \rightarrow X)$$

Segal types

When does (X_n) satisfies the Segal condition ?

$$(\Delta^n \rightarrow X) \xrightarrow{\sim} (\Delta_1 \amalg_{\Delta_0} \amalg \cdots \amalg_{\Delta_0} \Delta_1 \rightarrow X)$$



Complete types

By horn filling: a Segal type A admits composites $g \circ f$.

Complete types

By horn filling: a Segal type A admits composites $g \circ f$.

$$\text{iso}(f) := \left(\sum_{g:\text{hom}_A(y,x)} g \circ f = \text{id}_x \right) \times \left(\sum_{h:\text{hom}_A(y,x)} f \circ h = \text{id}_y \right)$$

Complete types

By horn filling: a Segal type A admits composites $g \circ f$.

$$\text{isiso}(f) := \left(\sum_{g:\text{hom}_A(y,x)} g \circ f = \text{id}_x \right) \times \left(\sum_{h:\text{hom}_A(y,x)} f \circ h = \text{id}_y \right)$$

$$x \simeq_A y := \sum_{f:\text{hom}_A(x,y)} \text{isiso}(f)$$

Complete types

By horn filling: a Segal type A admits composites $g \circ f$.

$$\text{isiso}(f) := \left(\sum_{g:\text{hom}_A(y,x)} g \circ f = \text{id}_x \right) \times \left(\sum_{h:\text{hom}_A(y,x)} f \circ h = \text{id}_y \right)$$

$$x \simeq_A y := \sum_{f:\text{hom}_A(x,y)} \text{isiso}(f)$$

$$\text{idtoiso} : \prod_{x,y:A} x =_A y \rightarrow x \cong_A y$$

Complete types

By horn filling: a Segal type A admits composites $g \circ f$.

$$\text{is iso}(f) := \left(\sum_{g:\text{hom}_A(y,x)} g \circ f = \text{id}_x \right) \times \left(\sum_{h:\text{hom}_A(y,x)} f \circ h = \text{id}_y \right)$$

$$x \simeq_A y := \sum_{f:\text{hom}_A(x,y)} \text{is iso}(f)$$

$$\text{id to iso} : \prod_{x,y:A} x =_A y \rightarrow x \cong_A y$$

Reminiscent to Rezk-completes/univalent categories in HoTT.

Hom Types in STT

In STT:

$$\mathrm{hom}_A(x, y) = \sum_{f: I \rightarrow A} (f\,0 = x) \times (f\,1 = y)$$

$$x \text{ ————— } y$$

Hom Types in STT

In STT:

$$\mathrm{hom}_A(x, y) = \sum_{f: I \rightarrow A} (f\,0 = x) \times (f\,1 = y)$$

$$x \text{ ————— } y$$

Then:

$$\mathrm{hom}_{\mathrm{hom}_A(x, y)}(f, g) = \sum_{H: I^2 \rightarrow A} \cdots$$

$$\begin{array}{ccc} x & \text{— } f \text{ —} & y \\ \parallel & & \parallel \\ x & \text{— } g \text{ —} & y \end{array}$$

The problem of hom-types

In l^2 : x, y invertible $\Rightarrow (x, y)$ too.

The problem of hom-types

In $\mathcal{I}^2$: x, y invertible $\Rightarrow (x, y)$ too.

$\mathcal{I}^2$ is 1-categorical: hom of a hom can only be a *groupoid* again.

The problem of hom-types

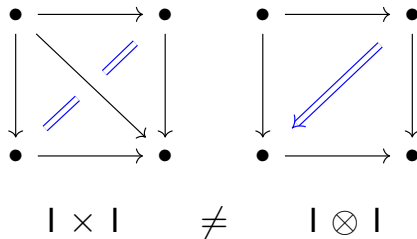
In $\mathcal{I}^2$: x, y invertible $\Rightarrow (x, y)$ too.

$\mathcal{I}^2$ is 1-categorical: hom of a hom can only be a *groupoid* again.

The problem of hom-types

In $\mathcal{I}^2$: x, y invertible $\Rightarrow (x, y)$ too.

$\mathcal{I}^2$ is 1-categorical: hom of a hom can only be a *groupoid* again.

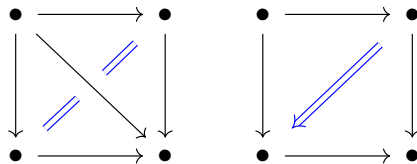


The problem of hom-types

In I^2 : x, y invertible $\Rightarrow (x, y)$ too.

I^2 is 1-categorical: hom of a hom can only be a *groupoid* again.

$$\text{hom}_A(x, y) \neq \sum_{f: I \rightarrow A} (f \cdot 0 = x) \times (f \cdot 1 = y)$$



$$I \times I \neq I \otimes I$$

Replacing $\rightarrow$

- Idea: replacing $(- \times I) \dashv (I \rightarrow -)$ by $(- \otimes I) \dashv (I \multimap -)$

Replacing $\rightarrow$

- Idea: replacing $(- \times I) \dashv (I \rightarrow -)$ by $(- \otimes I) \dashv (I \multimap -)$

$$I \multimap (I \multimap A) \simeq (I \otimes I \multimap A)$$

Replacing $\rightarrow$

- Idea: replacing $(- \times I) \dashv (I \rightarrow -)$ by $(- \otimes I) \dashv (I \rightarrow -)$

$$I \rightarrow (I \rightarrow A) \simeq (I \otimes I \rightarrow A)$$

- Extend contexts in two ways (bunched logic à la Pym–O’Hearn).

Replacing $\rightarrow$

- Idea: replacing $(- \times I) \dashv (I \rightarrow -)$ by $(- \otimes I) \dashv (I \multimap -)$

$$I \multimap (I \rightarrow A) \simeq (I \otimes I \rightarrow A)$$

- Extend contexts in two ways (bunched logic à la Pym–O’Hearn).
- Usual context extension: Γ, Δ

Replacing $\rightarrow$

- Idea: replacing $(- \times I) \dashv (I \rightarrow -)$ by $(- \otimes I) \dashv (I \rightarrow -)$

$$I \rightarrow (I \rightarrow A) \simeq (I \otimes I \rightarrow A)$$

- Extend contexts in two ways (bunched logic à la Pym–O’Hearn).
- Usual context extension: Γ, Δ
- Directed, semicartesian: $\Gamma \otimes \Delta$

Replacing $\rightarrow$

- Idea: replacing $(- \times I) \dashv (I \rightarrow -)$ by $(- \otimes I) \dashv (I \rightarrow -)$

$$I \rightarrow (I \rightarrow A) \simeq (I \otimes I \rightarrow A)$$

- Extend contexts in two ways (bunched logic à la Pym–O’Hearn).
- Usual context extension: Γ, Δ
- Directed, semicartesian: $\Gamma \otimes \Delta$

$$\bullet \xRightarrow{\quad} \bullet$$

$$\Gamma \otimes \Delta$$

$$\bullet \text{ — } \bullet$$

$$\Gamma, \Delta$$

Replacing $\rightarrow$

- Idea: replacing $(- \times I) \vdash (I \rightarrow -)$ by $(- \otimes I) \vdash (I \rightarrow -)$

$$I \rightarrow (I \rightarrow A) \simeq (I \otimes I \rightarrow A)$$

- Extend contexts in two ways (bunched logic à la Pym–O’Hearn).
- Usual context extension: Γ, Δ
- Directed, semicartesian: $\Gamma \otimes \Delta$

$$\bullet \xRightarrow{\quad} \bullet$$

$$\Gamma \otimes \Delta$$

$$\bullet \text{ — } \bullet$$

$$\Gamma, \Delta$$

$$\Gamma \otimes \Delta \neq \Delta \otimes \Gamma$$

Interpreted as the Crans-Gray tensor product.

Bunches

$$\Gamma, \Delta ::= \diamond \mid x : A \mid \Gamma \otimes \Delta \mid \Gamma, \Delta$$

Bunches

$$\Gamma, \Delta ::= \diamond \mid x : A \mid \Gamma \otimes \Delta \mid \Gamma, \Delta$$

Judgments live in a split context $\underline{\Lambda} \mid \Gamma$: a *crisp* part, and a *bunch*.

$$\underline{\Lambda} \mid \Gamma \vdash a : A$$

$$\underline{\Lambda} \mid \Gamma \vdash a \equiv b : A$$

Bunches

$$\Gamma, \Delta ::= \diamond \mid x : A \mid \Gamma \otimes \Delta \mid \Gamma, \Delta$$

Judgments live in a split context $\underline{\Lambda} \mid \Gamma$: a *crisp* part, and a *bunch*.

$$\underline{\Lambda} \mid \Gamma \vdash a : A \qquad \underline{\Lambda} \mid \Gamma \vdash a \equiv b : A$$

crisp assumptions does not care about functoriality.

Rules for $\rightarrow$ and $\multimap$

$$(\rightarrow\text{-INTRO}) \frac{\Gamma \otimes (x : A) \vdash b : B}{\Gamma \vdash \lambda x. b : A \rightarrow B}$$

Rules for $\rightarrow$ and $\multimap$

$$(\rightarrow\text{-INTRO}) \frac{\Gamma \otimes (x : A) \vdash b : B}{\Gamma \vdash \lambda x. b : A \rightarrow B}$$

$$(\rightarrow\text{-ELIM}) \frac{\Gamma \vdash f : A \rightarrow B \quad \Delta \vdash a : A}{\Gamma \otimes \Delta \vdash f \mid a \rangle : B}$$

Rules for $\rightarrow$ and $\multimap$

$$(\rightarrow\text{-INTRO}) \frac{\Gamma \otimes (x : A) \vdash b : B}{\Gamma \vdash \lambda x. b : A \rightarrow B}$$

$$(\multimap\text{-INTRO}) \frac{(x : A) \otimes \Gamma \vdash b : B}{\partial x. b : A \multimap B}$$

$$(\rightarrow\text{-ELIM}) \frac{\Gamma \vdash f : A \rightarrow B \quad \Delta \vdash a : A}{\Gamma \otimes \Delta \vdash f \mid a \rangle : B}$$

Rules for $\rightarrow$ and $\multimap$

$$(\rightarrow\text{-INTRO}) \frac{\Gamma \otimes (x : A) \vdash b : B}{\Gamma \vdash \lambda x. b : A \rightarrow B}$$

$$(\rightarrow\text{-ELIM}) \frac{\Gamma \vdash f : A \rightarrow B \quad \Delta \vdash a : A}{\Gamma \otimes \Delta \vdash f | a \rangle : B}$$

$$(\multimap\text{-INTRO}) \frac{(x : A) \otimes \Gamma \vdash b : B}{\partial x. b : A \multimap B}$$

$$(\multimap\text{-ELIM}) \frac{\Gamma \vdash a : A \quad \Delta \vdash f : A \multimap B}{\Gamma \otimes \Delta \vdash \langle a | f : B}$$

The $\otimes \dashv (\multimap, \rightarrow)$ adjunctions

For crisp types $X, Y, Z :: \mathcal{U}$

$$(X \otimes Y \rightarrow Z) \simeq (X \rightarrow Y \rightarrow Z) \quad (X \otimes Y \multimap Z) \simeq (Y \multimap X \multimap Z)$$

The $\otimes \dashv (\multimap, \rightarrow)$ adjunctions

For crisp types $X, Y, Z :: \mathcal{U}$

$$(X \otimes Y \rightarrow Z) \simeq (X \rightarrow Y \rightarrow Z) \quad (X \otimes Y \multimap Z) \simeq (Y \multimap X \multimap Z)$$

witnessed by currying

$$f \longmapsto \lambda x. \lambda y. f \mid x \otimes y \rangle \quad f \longmapsto \partial x. \partial y. \langle x \otimes y \mid f$$

The $\otimes \dashv (\multimap, \rightarrow)$ adjunctions

For crisp types $X, Y, Z :: \mathcal{U}$

$$(X \otimes Y \rightarrow Z) \simeq (X \rightarrow Y \rightarrow Z) \quad (X \otimes Y \multimap Z) \simeq (Y \multimap X \multimap Z)$$

witnessed by currying

$$f \longmapsto \lambda x. \lambda y. f \mid x \otimes y \rangle$$

$$f \longmapsto \lambda x. \lambda y. \langle x \otimes y \mid f$$

$$\lambda (x \otimes y). f \mid x \rangle \mid y \rangle \longleftarrow f$$

$$\lambda (x \otimes y). \langle x \mid \langle y \mid f \longleftarrow f$$

The $\otimes \dashv (\multimap, \multimap)$ adjunctions

For crisp types $X, Y, Z :: \mathcal{U}$

$$(X \otimes Y \multimap Z) \simeq (X \multimap Y \multimap Z) \quad (X \otimes Y \multimap Z) \simeq (Y \multimap X \multimap Z)$$

witnessed by currying

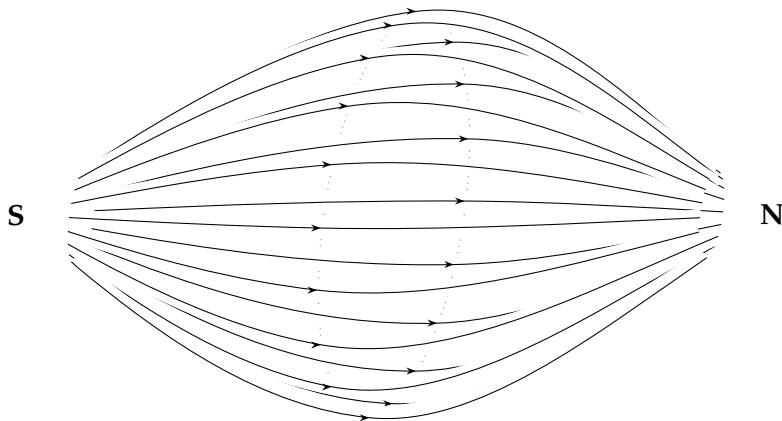
$$f \longmapsto \lambda x. \lambda y. f \mid x \otimes y \rangle \quad f \longmapsto \lambda x. \lambda y. \langle x \otimes y \mid f$$

$$\lambda (x \otimes y). f \mid x \rangle \mid y \rangle \longleftarrow f \quad \lambda (x \otimes y). \langle x \mid \langle y \mid f \longleftarrow f$$

$- \otimes Y$ is left adjoint to $Y \multimap -$

$Y \otimes -$ is left adjoint to $Y \multimap -$

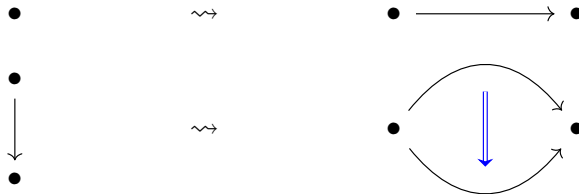
Suspension



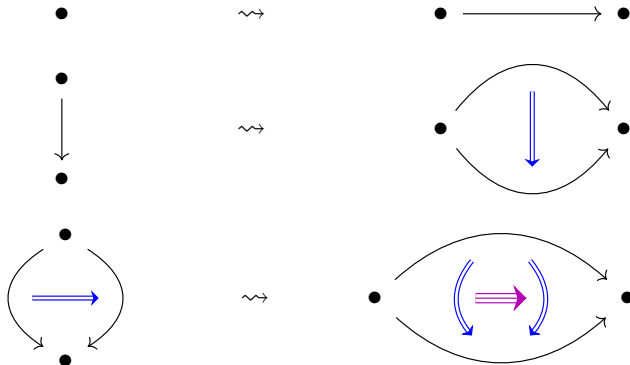
Suspension – Examples



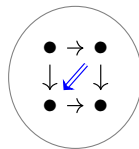
Suspension – Examples



Suspension – Examples

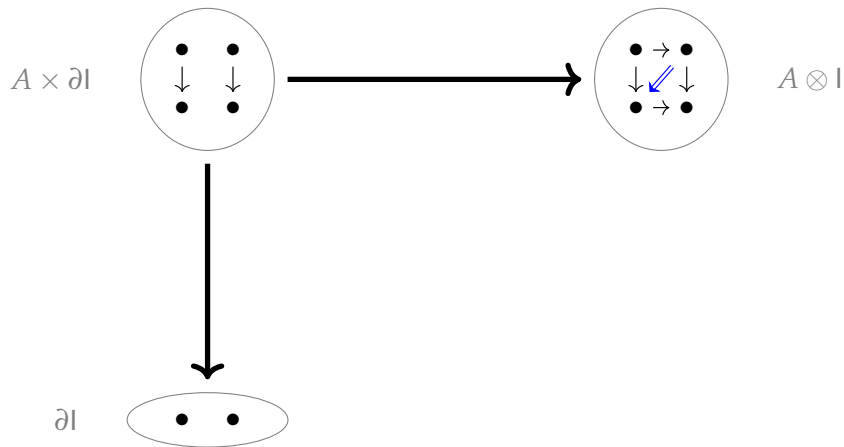


Suspension from $\otimes$

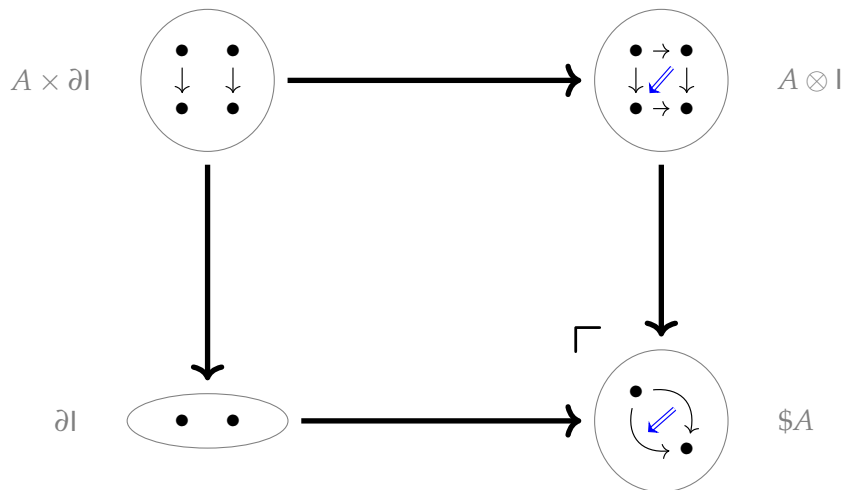


$$A \otimes I$$

Suspension from $\otimes$



Suspension from $\otimes$



Suspension $\dashv$ hom

suspension $\$A$, as a pushout:

$$\begin{array}{ccc} A \times \partial I & \longrightarrow & A \otimes I \\ \downarrow & \lrcorner & \downarrow \\ \partial I & \longrightarrow & \$A \end{array}$$

Suspension $\dashv$ hom

suspension $\$A$, as a pushout:

$$\begin{array}{ccc} A \times \partial I & \longrightarrow & A \otimes I \\ \downarrow & \lrcorner & \downarrow \\ \partial I & \longrightarrow & \$A \end{array}$$

$$\mathrm{hom}_A(x, y) \equiv \sum_{f: I \rightarrow A} (f \mid 0\rangle = x) \times (f \mid 1\rangle = y)$$

Suspension $\dashv$ hom

suspension $\$A$, as a pushout:

$$\begin{array}{ccc} A \times \partial I & \longrightarrow & A \otimes I \\ \downarrow & \lrcorner & \downarrow \\ \partial I & \longrightarrow & \$A \end{array}$$

$$\mathrm{hom}_A(x, y) \equiv \sum_{f: I \rightarrow A} (f|_0 = x) \times (f|_1 = y)$$

Theorem

$$(\$X \rightarrow Y \text{ sending } 0, 1 \text{ to } y_0, y_1) \simeq (X \rightarrow \mathrm{hom}_Y(y_0, y_1))$$

batt: a type-checker for BaTT

 $/\backslash^{\cdot} _ \cdot^{\cdot} / \backslash$

- Ongoing Ocaml implementation by S. Mimram:
`github.com/smimram/batt`

$$\wedge \vee \cdot _ \cdot \wedge \wedge$$

- A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

batt: a type-checker for BaTT

/ \ ^ . _ . ^ / \

- Ongoing Ocaml implementation by S. Mimram:
`github.com/smimram/batt`
- Lexer/parser, Type-checker, Metavariables, Unification.
- Small module system, Holes à la Agda, Readable error messages !

Examples

`smimram.github.io/batt/`

The end

Thank you !

Questions ?

